

COMPSCI 1DM3 - Winter 2026

Discrete Mathematics for Computer Science

Angela Zavaleta Bernuy



Midterm Information



Angela Zavaleta Bernuy posted on Feb 10, 2026 12:16 PM

 Edited

The midterm for this course will be on Monday, February 23rd at 7pm in PGCLL B138. The midterm will be on paper and will last 2 hours.

The midterm will cover the sections of chapter 1, 2, and 3 that were covered in class. To prepare for the test, you should review the problem sets, quizzes and assignments. However, we recognize that some of those problems can be too complex to solve in a short period of time. We have put together this list of problems that are similar in complexity to what you can expect on the midterm (see attachment).

No aids are allowed, however, we will provide you with the list of rules of inference, some summation formulas, and some set properties. The tables are added at the end of the practice problems for the midterm file.

Attachment(s):

 [Midterm Practice Problems...](#)

(474.82 KB)

TABLE 1 Set Identities.	
<i>Identity</i>	<i>Name</i>
$A \cap U = A$ $A \cup \emptyset = A$	Identity laws
$A \cup U = U$ $A \cap \emptyset = \emptyset$	Domination laws
$A \cup A = A$ $A \cap A = A$	Idempotent laws
$\overline{(\overline{A})} = A$	Complementation law
$A \cup B = B \cup A$ $A \cap B = B \cap A$	Commutative laws
$A \cup (B \cap C) = (A \cup B) \cap C$ $A \cap (B \cup C) = (A \cap B) \cup C$	Associative laws
$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$ $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$	Distributive laws
$\overline{A \cap B} = \overline{A} \cup \overline{B}$ $\overline{A \cup B} = \overline{A} \cap \overline{B}$	De Morgan's laws
$A \cup (A \cap B) = A$ $A \cap (A \cup B) = A$	Absorption laws
$A \cup \overline{A} = U$ $A \cap \overline{A} = \emptyset$	Complement laws

TABLE 1 Rules of Inference.		
<i>Rule of Inference</i>	<i>Tautology</i>	<i>Name</i>
$\begin{array}{l} p \\ p \rightarrow q \\ \hline \therefore q \end{array}$	$(p \wedge (p \rightarrow q)) \rightarrow q$	Modus ponens
$\begin{array}{l} \neg q \\ p \rightarrow q \\ \hline \therefore \neg p \end{array}$	$(\neg q \wedge (p \rightarrow q)) \rightarrow \neg p$	Modus tollens
$\begin{array}{l} p \rightarrow q \\ q \rightarrow r \\ \hline \therefore p \rightarrow r \end{array}$	$((p \rightarrow q) \wedge (q \rightarrow r)) \rightarrow (p \rightarrow r)$	Hypothetical syllogism
$\begin{array}{l} p \vee q \\ \neg p \\ \hline \therefore q \end{array}$	$((p \vee q) \wedge \neg p) \rightarrow q$	Disjunctive syllogism
$\begin{array}{l} p \\ \hline \therefore p \vee q \end{array}$	$p \rightarrow (p \vee q)$	Addition
$\begin{array}{l} p \wedge q \\ \hline \therefore p \end{array}$	$(p \wedge q) \rightarrow p$	Simplification
$\begin{array}{l} p \\ q \\ \hline \therefore p \wedge q \end{array}$	$((p) \wedge (q)) \rightarrow (p \wedge q)$	Conjunction
$\begin{array}{l} p \vee q \\ \neg p \vee r \\ \hline \therefore q \vee r \end{array}$	$((p \vee q) \wedge (\neg p \vee r)) \rightarrow (q \vee r)$	Resolution

TABLE 6 Logical Equivalences.	
<i>Equivalence</i>	<i>Name</i>
$p \wedge \mathbf{T} \equiv p$ $p \vee \mathbf{F} \equiv p$	Identity laws
$p \vee \mathbf{T} \equiv \mathbf{T}$ $p \wedge \mathbf{F} \equiv \mathbf{F}$	Domination laws
$p \vee p \equiv p$ $p \wedge p \equiv p$	Idempotent laws
$\neg(\neg p) \equiv p$	Double negation law
$p \vee q \equiv q \vee p$ $p \wedge q \equiv q \wedge p$	Commutative laws
$(p \vee q) \vee r \equiv p \vee (q \vee r)$ $(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$	Associative laws
$p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$ $p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$	Distributive laws
$\neg(p \wedge q) \equiv \neg p \vee \neg q$ $\neg(p \vee q) \equiv \neg p \wedge \neg q$	De Morgan's laws
$p \vee (p \wedge q) \equiv p$ $p \wedge (p \vee q) \equiv p$	Absorption laws
$p \vee \neg p \equiv \mathbf{T}$ $p \wedge \neg p \equiv \mathbf{F}$	Negation laws

TABLE 2 Some Useful Summation Formulae.

<i>Sum</i>	<i>Closed Form</i>
$\sum_{k=0}^n ar^k \ (r \neq 0)$	$\frac{ar^{n+1} - a}{r - 1}, r \neq 1$
$\sum_{k=1}^n k$	$\frac{n(n+1)}{2}$
$\sum_{k=1}^n k^2$	$\frac{n(n+1)(2n+1)}{6}$
$\sum_{k=1}^n k^3$	$\frac{n^2(n+1)^2}{4}$
$\sum_{k=0}^{\infty} x^k, x < 1$	$\frac{1}{1-x}$
$\sum_{k=1}^{\infty} kx^{k-1}, x < 1$	$\frac{1}{(1-x)^2}$

Last Lecture

- (Asymptotic upper bound:) We say $f(x)$ is $O(g(x))$ if for all $x \geq k$,

$$|f(x)| \leq C \cdot |g(x)|$$

- (Asymptotic lower bound:) We say $f(x)$ is $\Omega(g(x))$ if for all $x \geq k$.

$$|f(x)| \geq C \cdot |g(x)|,$$

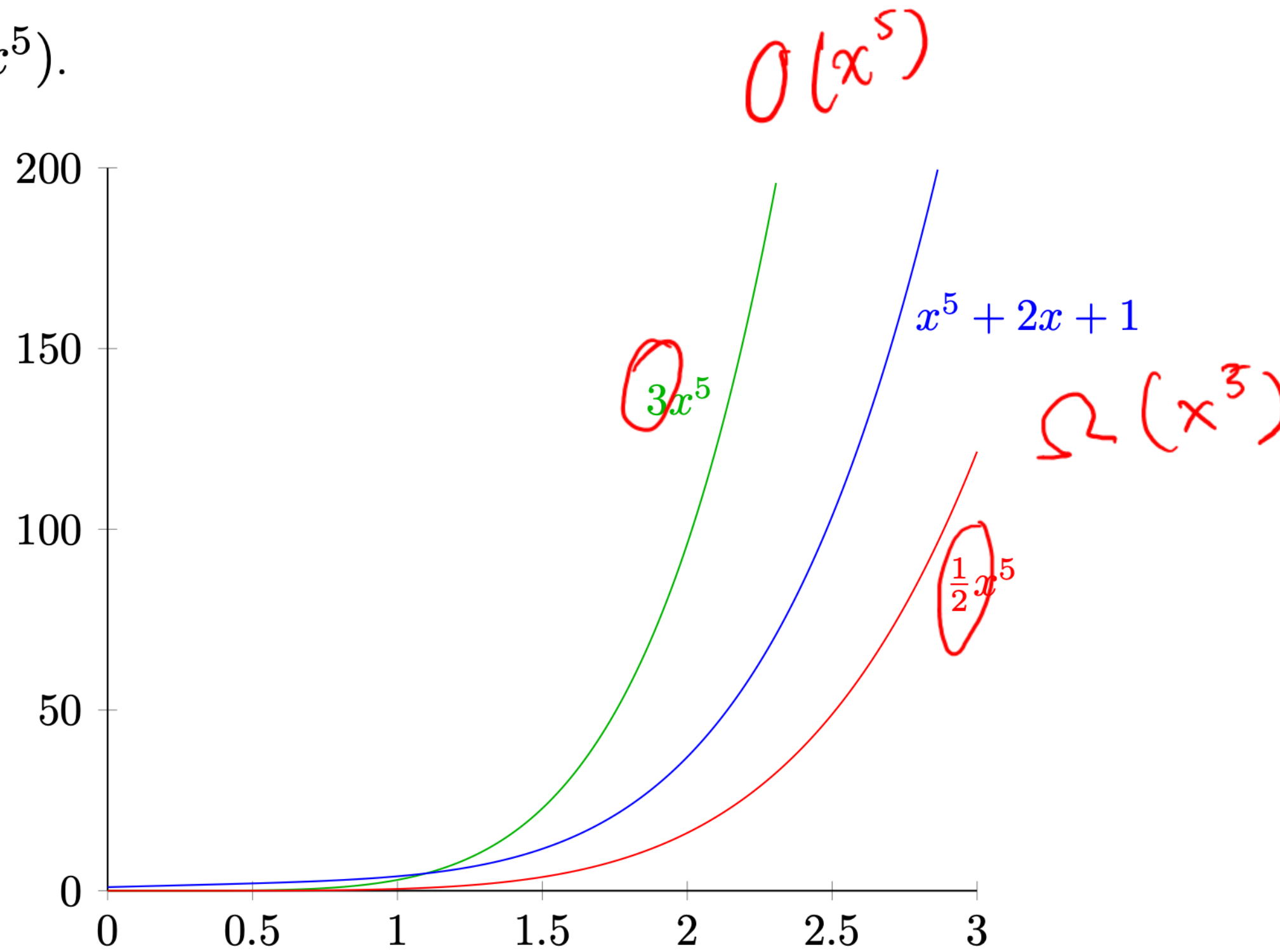
- We say $f(x)$ is $\Theta(g(x))$ if $f(x)$ is both $O(g(x))$ and $\Omega(g(x))$. Equivalently, if for all $x \geq k$,

$$C_1|g(x)| \leq |f(x)| \leq C_2|g(x)|$$

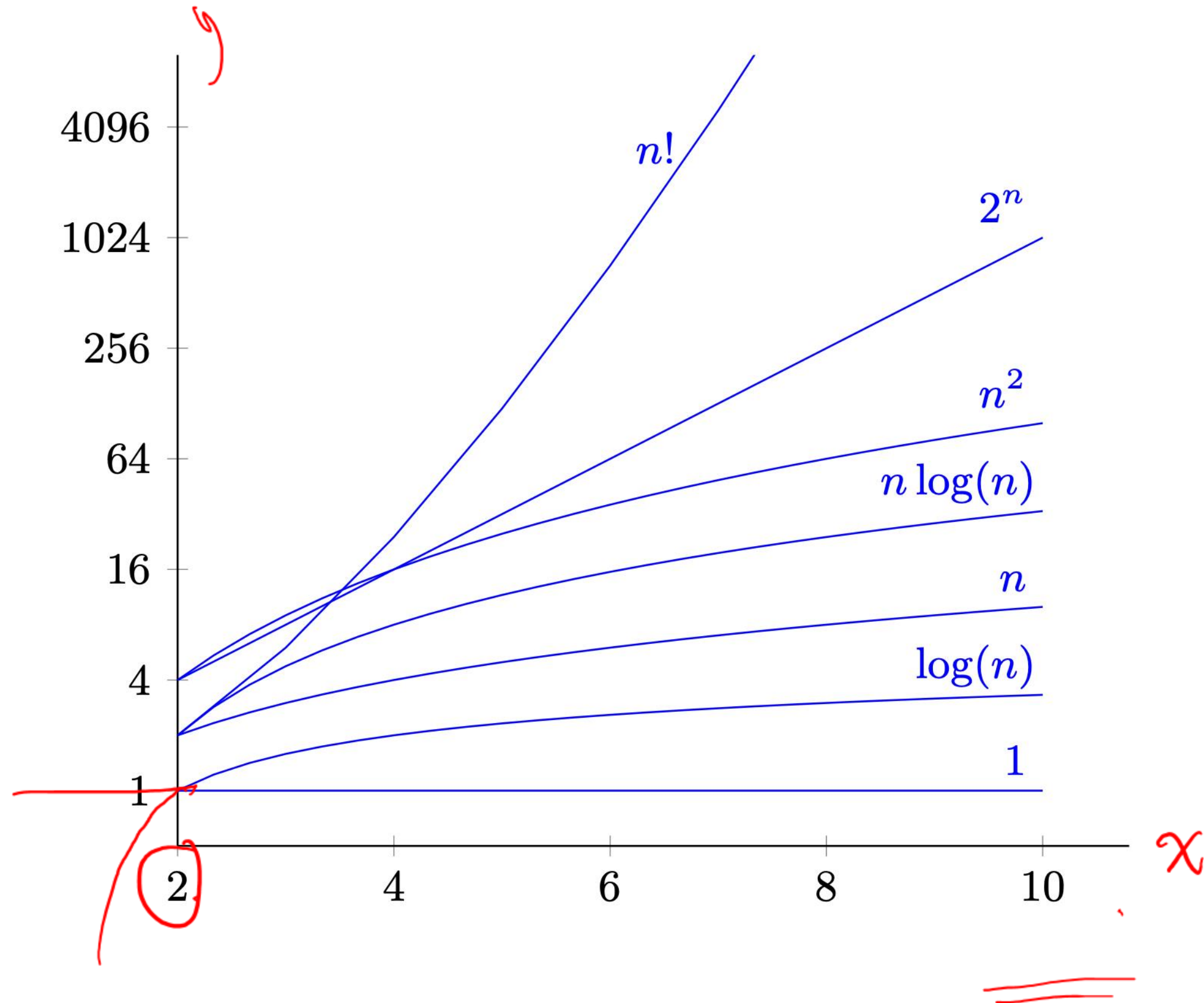
- All polynomials of degree n are $O(x^n)$.

Example: Θ

$x^5 + 2x + 1$ is $\Theta(x^5)$.



Example: growth of functions



Examples

$$f(x) = O(x^n), \quad f(x) = \Omega(x^n)$$

Theorem

Let f be a polynomial function of degree n , that is,

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + a_{n-2} x^{n-2} + \cdots + a_1 x + a_0,$$

for some constants $a_n, a_{n-1}, \dots, a_1, a_0$. Then, we have $f(x) = \Theta(x^n)$.

for $x \geq 1$

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

$$\geq \underbrace{a_n}_{c} x^n \rightarrow \Omega(x^n)$$

$$\therefore f(x) = \Theta(x^n)$$

Sec 3.3: Complexity of Algorithms

Main questions:

- How to compare two algorithms?
- How to define “efficient algorithms”?

There are typically two measures:

- Space complexity: the storage capacity required for executing a given algorithm
- Time complexity: the time required for running a given algorithm. Also known as run-time complexity

Note: We measure time complexity of an algorithm in terms of the **number of operations** performed by the algorithm. Why?

Complexity analysis

- **Worst-case** complexity: the largest number of operations used to solve the problem over all possible inputs of a given size.
- **average-case** complexity: the average number of operations used to solve the problem over all possible inputs of a given size.

Example

Find the time complexity of maximum-finding algorithm:

```
1 function find_max(L, n):  
2    $C_1$  max_value = L[1]  $\rightarrow O(1)$   
3   for i = 2 to n do  
4     if L[i] > max_value then  $\rightarrow$  comparison  $O(1) - C_2$   
5       max_value = L[i]  $\rightarrow$  assignment  $O(1) - C_3$   
6 return max_value
```

2 to n times $\rightarrow n-1$ times

$\rightarrow \Theta(n) \rightarrow O(n) \& \Omega(n)$ linear

of operations

$$T(n) = C_1 + (n-1)(C_2 + C_3)$$

$$\approx an + b$$

Example

Find the time complexity of linear search algorithm:

```
1 function LinearSearch(L, n, target):  
2     for i = 1 to n do  
3         if L[i] = target then  
4             return i  
5     return -1 # Target not found
```

n times.
→ n comp.

Best case
 $L[1] == \text{target}$.
 $\theta(1)$

Worst case.
 $L[n] == \text{target}$
or
not found
 $\theta(n)$

Avg case
run loop $n/2$
 $\frac{1}{2} \cdot n$
 $\theta(n)$

Example

problem size $n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \dots$

Find the time complexity of binary search algorithm:

```
1 function BinarySearch(L, n, target):  
2     left = 1  
3     right = n  
4     while left <= right do  
5         mid = floor((left + right) / 2)  
6         if L[mid] = target then  
7             return mid  
8         if L[mid] < target then  
9             left = mid + 1  
10        else  
11            right = mid - 1  
12    return -1 # Target not found
```

After k iterations.

problem size of
 $\frac{n}{2^k}$

loop stop. $\frac{n}{2^k} \leq 1$
 $k \approx \log_2 n$

$\Theta(\log n)$

Example

sorted or unsorted
unknown.

Find the time complexity of insertion sort algorithm:

```
1 function InsertionSort(L, n):  
2   for i = 2 to n do  
3     key = L[i]  
4     j = i - 1  
5     while j >= 1 and L[j] > key do  
6       L[j + 1] = L[j]  
7       j = j - 1  
8     L[j + 1] = key
```

n times { }

Worst case (reversed list)
for i, while loop. i-1
of iteration of inner loop.
$$\sum_{i=2}^n (i-1) = \frac{n(n-1)}{2} \Rightarrow \theta(n^2)$$

Avg case:

inner loop: $\frac{(i-1)}{2}$

$\Rightarrow \theta(n^2)$

Best case
sorted list.

while - constant time
(won't run)

$\theta(n)$

Complexity terminology

Complexity	Terminology
$\Theta(1)$	<u>Constant</u> complexity
$\Theta(\log n)$	Logarithmic complexity
$\Theta(n)$	<u>Linear</u> complexity
$\Theta(n \log n)$	<u>Linearithmic</u> complexity
$\Theta(n^b)$	<u>Polynomial</u> complexity
$\Theta(b^n)$, where $b > 1$	<u>Exponential</u> complexity
$\Theta(n!)$	<u>Factorial</u> complexity